

-- Q1: What's the overall transaction failure rate, and how much value is lost to failures?

```
SELECT
  COUNT(*) AS total_transactions,
  SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) AS failed_transactions,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS failed_rate,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN AMOUNT ELSE 0 END), 2) AS failed_transaction_value
FROM fact_transactions;
```

#	⚙ TOTAL_TRANSACTIONS	⚙ FAILED_TRANSACTIONS	⚙ FAILED_RATE	⚙ FAILED_TRANSACTION_VALUE
1	300000	12007	4.00	141295843.02

-- Q2: Which specific failure reasons are most common, and which cost the most money?

```
CREATE OR REPLACE VIEW VW_FAILURE_BY_REASON AS
SELECT
  PAYMENT_STATUS,
  REASON,
  COUNT(*) AS failure_count,
  ROUND(SUM(AMOUNT), 2) AS value_lost,
  ROUND(AVG(AMOUNT), 2) AS avg_value_lost
FROM fact_transactions
WHERE PAYMENT_STATUS != 'Successful'
GROUP BY PAYMENT_STATUS, REASON
ORDER BY failure_count DESC;
```

SELECT * FROM VW_FAILURE_BY_REASON;

#	⚙ PAYMENT_STATUS	⚙ REASON	⚙ FAILURE_COUNT	⚙ VALUE_LOST	⚙ AVG_VALUE_LOST
1	Failed	Server error	3361	13192378.35	3925.13
2	Failed	Wrong PIN	3315	13231829.62	3991.5
3	Failed	Insufficient amount	3304	12893391.45	3902.36
4	Wrong PIN	Wrong Info	700	35201660.59	50288.09
5	Server error	Server error	692	34554471.9	49934.21
6	Insufficient amount	Bank Denied	635	32222111.11	50743.48

-- Q3: Which service (Money Transfer, Recharge/Bills, Loans, Insurance) is least reliable?

```
CREATE OR REPLACE VIEW VW_FAILURE_BY_SERVICE AS
SELECT
  SERVICE,
  COUNT(*) AS total_transactions,
  SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) AS failed_count,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) as failed_rate,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN AMOUNT ELSE 0 END), 2) AS failed_transaction_value,
FROM fact_transactions
GROUP BY SERVICE
ORDER BY failed_rate DESC;
```

SELECT * FROM VW_FAILURE_BY_SERVICE;

#	⚙ SERVICE	⚙ TOTAL_TRANSACTIONS	⚙ FAILED_COUNT	⚙ FAILED_RATE	⚙ FAILED_TRANSACTION_VALUE
1	Insurance	50000	2124	4.25	22207163.04
2	Loans	50000	2027	4.05	101978243.6
3	Money_Transfer	150000	5935	3.96	15124080.59
4	Recharge_Bills	50000	1921	3.84	1986355.79

--Q4: Drill deeper — which specific Service Type (UPI, QR Code, Mobile Recharge, etc.) fails most?

```
CREATE OR REPLACE VIEW VW_FAILURE_BY_SERVICE_TYPE AS
SELECT
  SERVICE,
  'SERVICE TYPE',
  COUNT(*) AS total_transactions,
  SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) AS failed_count,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS failure_rate_pct
FROM FACT_TRANSACTIONS
GROUP BY SERVICE, 'SERVICE TYPE'
ORDER BY failure_rate_pct DESC;
```

SELECT * FROM VW_FAILURE_BY_SERVICE_TYPE;

-- Q5: Do larger transactions fail more often than smaller ones?

```
CREATE OR REPLACE VIEW VW_FAILURE_BY_AMOUNT_BUCKET AS
SELECT
  CASE
    WHEN AMOUNT < 500 THEN '0-500'
    WHEN AMOUNT < 1000 THEN '500-1000'
    WHEN AMOUNT < 5000 THEN '1000-5000'
    WHEN AMOUNT < 20000 THEN '5000-20000'
    ELSE '20000+'
  END AS amount_bucket,
  COUNT(*) AS total_transactions,
  SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) AS failed_count,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END)*100.0 / COUNT(*), 2) AS failed_rate
FROM FACT_TRANSACTIONS
GROUP BY amount_bucket
ORDER BY
  CASE amount_bucket
    WHEN '0-500' THEN 1
    WHEN '500-1000' THEN 2
```

```
WHEN '1000-5000' THEN 3
WHEN '5000-20000' THEN 4
ELSE 5 END;
```

SELECT * FROM VW_FAILURE_BY_AMOUNT_BUCKET;

#	AMOUNT_BUCKET	TOTAL_TRANSACTIONS	FAILED_COUNT	FAILED_RATE
1	0-500	25965	986	3.80
2	500-1000	28746	1121	3.90
3	1000-5000	158814	6308	3.97
4	5000-20000	45994	1992	4.33
5	20000+	40481	1600	3.95

-- Q6: Is there a day-of-week pattern in failures? (e.g., weekend server strain)

```
CREATE OR REPLACE VIEW VW_FAILURE_BY_DAY_OF_WEEK AS
SELECT
  DAYNAME(DATE) AS day_of_week,
  COUNT(*) AS total_transactions,
  SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) AS failed_count,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS failed_rate
FROM fact_transactions
GROUP BY day_of_week
ORDER BY failed_rate;
```

SELECT * FROM VW_FAILURE_BY_DAY_OF_WEEK;

#	DAY_OF_WEEK	TOTAL_TRANSACTIONS	FAILED_COUNT	FAILED_RATE
1	Mon	43442	1708	3.93
2	Sun	42842	1699	3.97
3	Wed	42985	1709	3.98
4	Fri	42264	1697	4.02
5	Thu	42898	1730	4.03
6	Tue	43223	1747	4.04
7	Sat	42346	1717	4.05

-- Q7: Is failure rate trending up or down month over month?

```
CREATE OR REPLACE VIEW VW_FAILURE_TREND_MONTHLY AS
SELECT
  DATE_TRUNC('MONTH', DATE)::DATE AS month,
  COUNT(*) AS total_transactions,
  SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) AS failed_count,
  ROUND(SUM(CASE WHEN PAYMENT_STATUS != 'Successful' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS failed_rate
FROM fact_transactions
GROUP BY month
ORDER BY month;
```

SELECT * FROM VW_FAILURE_TREND_MONTHLY;

#	MONTH	TOTAL_TRANSACTIONS	FAILED_COUNT	FAILED_RATE
1	2024-01-01	25428	951	3.74
2	2024-02-01	24040	946	3.94
3	2024-03-01	25306	1058	4.18
4	2024-04-01	24563	924	3.76
5	2024-05-01	25659	1040	4.05
6	2024-06-01	24587	994	4.04
7	2024-07-01	25576	1076	4.21
8	2024-08-01	25329	965	3.81
9	2024-09-01	24507	995	4.06
10	2024-10-01	25478	1042	4.09
11	2024-11-01	24832	1022	4.12
12	2024-12-01	24695	994	4.03

-- Q8: Building the base cohort table

```
CREATE OR REPLACE VIEW VW_USER_COHORT_BASE AS
SELECT
  USER_ID,
  JOIN_DATE,
  DATE_TRUNC('MONTH', JOIN_DATE)::DATE AS cohort_month,
  (SELECT MAX(DATE) FROM fact_transactions) AS data_max_date,
  CASE
    WHEN JOIN_DATE > (SELECT MAX(DATE) FROM fact_transactions) THEN 1 ELSE 0
  END AS is_censored_cohort,
  GREATEST(DATEDIFF('MONTH', JOIN_DATE, (SELECT MAX(DATE) FROM fact_transactions)), 0) AS observation_month
FROM dim_users;
```

SELECT * FROM VW_USER_COHORT_BASE LIMIT 5 ;

#	USER_ID	JOIN_DATE	COHORT_MONTH	DATA_MAX_DATE	IS_CENSORED_COHORT	OBSERVATION_MONTH
1	PP0000001	2025-06-17	2025-06-01	2024-12-30	1	0
2	PP0000002	2023-12-10	2023-12-01	2024-12-30	0	12
3	PP0000003	2024-09-14	2024-09-01	2024-12-30	0	3
4	PP0000004	2023-10-19	2023-10-01	2024-12-30	0	14
5	PP0000005	2025-03-10	2025-03-01	2024-12-30	1	0

-- Q9: How many users fall in each cohort month, and what % are censored (2025 data-boundary issue)

```
SELECT
    COHORT_MONTH,
    COUNT(*) AS total_users,
    SUM(IS_CENSORED_COHORT) AS censored_users,
    ROUND(SUM(IS_CENSORED_COHORT) * 100.0 / COUNT(*), 2) AS pct_censored
FROM VW_USER_COHORT_BASE
GROUP BY COHORT_MONTH
ORDER BY COHORT_MONTH;
```

#	COHORT_MONTH	# TOTAL_USERS	# CENSORED_USERS	# PCT_CENSORED
1	2023-08-01	4314	0	0.00
2	2023-09-01	4495	0	0.00
3	2023-10-01	4543	0	0.00
4	2023-11-01	4395	0	0.00
5	2023-12-01	4593	0	0.00
6	2024-01-01	4523	0	0.00
7	2024-02-01	4386	0	0.00
8	2024-03-01	4487	0	0.00

-- Q10: Building the transaction-to-cohort link table

```
CREATE OR REPLACE VIEW VW_TXT_WITH_COHORT AS
SELECT
    t.TRANSACTION_ID,
    t.USER_ID,
    t.AMOUNT,
    t.DATE,
    t.PAYMENT_STATUS,
    u.cohort_month,
    u.is_censored_cohort,
    DATEDIFF('MONTH', u.join_date, t.DATE) AS month_since_join
FROM FACT_TRANSACTIONS t
JOIN VW_USER_COHORT_BASE u
ON t.USER_ID = u.USER_ID
WHERE DATEDIFF('MONTH', u.join_date, t.DATE) >= 0;
```

```
SELECT * FROM VW_TXT_WITH_COHORT;
```

#	TRANSACTION_ID	USER_ID	# AMOUNT	DATE	PAYMENT_STATUS	COHORT_MONTH	# IS_CENSORED_COHORT	# MONTH_SINCE_JOIN
1	RCG_92AE572161FA	PP1085390	1954.71	2024-11-20	Successful	2023-12-01	0	11
2	RCG_834ECCB39D81	PP1050343	1805.79	2024-08-25	Successful	2024-03-01	0	5
3	RCG_E7C3CD65734C	PP0029000	1373.12	2024-09-14	Successful	2023-08-01	0	13
4	RCG_A8DA9CEAC215	PP0026380	1750.47	2024-07-05	Successful	2024-04-01	0	3
5	RCG_5EB7F7B90C36	PP1089425	1615.3	2024-11-30	Successful	2024-06-01	0	5
6	RCG_40D03246A178	PP1053043	1070.83	2024-05-01	Successful	2024-05-01	0	0

-- Q11: The actual cohort retention matrix — % of each cohort still transacting in month 0, 1, 2, 3...

```
CREATE OR REPLACE VIEW VW_COHORT_RETENTION_MATRIX AS
WITH cohort_size AS (
    SELECT cohort_month, COUNT(DISTINCT USER_ID) AS cohort_users
    FROM VW_USER_COHORT_BASE
    WHERE is_censored_cohort = 0
    GROUP BY cohort_month
),
active_by_month AS (
    SELECT
        cohort_month,
        month_since_join,
        COUNT(DISTINCT user_id) AS active_user
    FROM VW_TXT_WITH_COHORT
    WHERE is_censored_cohort = 0
    GROUP BY cohort_month, month_since_join
)
SELECT
    a.cohort_month,
    a.month_since_join,
    c.cohort_users,
    a.active_user,
    ROUND(a.active_user * 100.0 / c.cohort_users, 2) AS retention_rate
FROM active_by_month a
JOIN cohort_size c
ON a.cohort_month = c.cohort_month
ORDER BY a.cohort_month, a.month_since_join;
```

```
SELECT * FROM VW_COHORT_RETENTION_MATRIX ORDER BY cohort_month, month_since_join;
```

#	COHORT_MONTH	# MONTH_SINCE_JOIN	# COHORT_USERS	# ACTIVE_USER	# RETENTION_RATE
1	2023-08-01	5	4314	908	21.05
2	2023-08-01	6	4314	831	19.26
3	2023-08-01	7	4314	947	21.95
4	2023-08-01	8	4314	922	21.37
5	2023-08-01	9	4314	883	20.47
6	2023-08-01	10	4314	869	20.17

-- Q12: Which join-month cohort has the best long-term retention?

```
SELECT
  cohort_month,
  MAX(CASE WHEN month_since_join = 0 THEN RETENTION_RATE END) AS month_0,
  MAX(CASE WHEN month_since_join = 3 THEN RETENTION_RATE END) AS month_3,
  MAX(CASE WHEN month_since_join = 6 THEN RETENTION_RATE END) AS month_6,
  MAX(CASE WHEN month_since_join = 12 THEN RETENTION_RATE END) AS month_12
FROM VW_COHORT_RETENTION_MATRIX
GROUP BY cohort_month
ORDER BY cohort_month;
```

	COHORT_MONTH	MONTH_0	MONTH_3	MONTH_6	MONTH_12
1	2023-08-01	null	null	19.26	20.89
2	2023-09-01	null	null	20.22	20.82
3	2023-10-01	null	21.55	20.89	21.40
4	2023-11-01	null	18.68	20.93	20.07
5	2023-12-01	null	20.90	20.51	20.14
6	2024-01-01	20.21	20.00	20.25	20.10

-- Q13: Retention priority — users who transacted early (month 0-1) but then went inactive (potential re-engagement targets)

```
CREATE OR REPLACE VIEW VW_DORMANT_USERS AS
SELECT DISTINCT USER_ID
FROM VW_TXT_WITH_COHORT
WHERE MONTH_SINCE_JOIN <= 1
AND USER_ID NOT IN (SELECT USER_ID FROM VW_TXT_WITH_COHORT WHERE MONTH_SINCE_JOIN >=3);
```

```
SELECT COUNT(*) FROM VW_DORMANT_USERS;
```

	COUNT(*)
1	10073